

**StudyMate AI – An Agentic AI-Powered Personalized Academic
Study Assistant**

**PROJECT REPORT FOR THE COURSE
AD23731 FOUNDATIONS OF AGENTIC AI**

Submitted by

TARUN AANAND S G (Reg. No. 231501172)

TARUN A D (Reg. No. 231501171)

SAI SENTHIL M (Reg. No. 231501140)



**DEPARTMENT OF
ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING
RAJALAKSHMI ENGINEERING COLLEGE
THANDALAM, CHENNAI, 602105**



RAJALAKSHMI ENGINEERING COLLEGE

Approved by AICTE | Affiliated to Anna University | Accredited by NAAC

BONAFIDE CERTIFICATE

Certified that this is a Bonafide Record of the project work done by **TARUN AANAND S G (231501172)**, **TARUN A D (231501171)** and **SAI SENTHIL M (231501140)** of VII Semester, B.Tech. Artificial Intelligence and Machine Learning Department in partial fulfilment of the requirements of the course **AD23731 – Foundations of Agentic AI** during the Academic Year 2026 – 2027.

Faculty in-charge

Submitted for the Project Review held on:

INTERNAL EXAMINER

EXTERNAL EXAMINER

Table of Contents

1. INTRODUCTION	1
1.1 Background and Motivation	1
1.2 Need for Agentic AI	1
1.3 Project Objectives	1
2. PROBLEM STATEMENT AND BUSINESS CASE	2
2.1 Problem Definition	2
2.2 Business and Real-World Context	2
2.3 Business Case	3
3. USER AND STAKEHOLDER ANALYSIS	3
3.1 Target Users	3
3.2 Stakeholder Identification	3
3.3 User Requirements	4
4. PROPOSED AGENTIC AI SOLUTION	4
4.1 Overall Solution	4
4.2 Key Features	5
4.3 Agent-Based Approach	5
5. DATASET / KNOWLEDGE SOURCES	6
5.1 Student-Provided Course Materials	6
5.2 Synthetic Evaluation Knowledge Source	6
5.3 Knowledge-Source Processing	6
5.4 Dataset / Knowledge-Source Limitation	7
6. TOOLS AND TECHNOLOGIES USED	7
7. AGENT ARCHITECTURE	8
7.1 Agent Components	8
7.2 Tools	8
7.3 Memory and State	8
7.4 Reasoning Flow	8
7.5 Agent Interactions	8
8. AGENT WORKFLOW AND ORCHESTRATION	8
8.1 Workflow Design	8
8.2 Task Decomposition	9
8.3 Agent Coordination	9
8.4 Orchestration Implementation	9
9. AGENTIC RAG IMPLEMENTATION	10
9.1 Retrieval Architecture	10
9.2 Knowledge Retrieval	10
9.3 Generation Process	11
9.4 Critique and Grounding Check	11
9.5 Retrieval-Quality Evaluation	11
10. SYSTEM IMPLEMENTATION	11
10.1 Functional Prototype	11
10.2 AI Advisor and Materials	11
10.3 Profile, Progress and Memory	12
10.4 Study Planner and Quiz	12
10.5 Source-Code Repository	12

10.6 Interface Representation	12
11. EXPERIMENTAL EVALUATION AND RESULTS	13
11.1 Evaluation Metrics	13
11.2 Testing Methodology	13
11.3 Performance and Functional Results	14
11.4 Success-Metric Achievement	14
12. COMPARISON, INNOVATION AND SCALABILITY	15
12.1 Comparison with Baseline	15
12.2 Advantages of the Proposed System	15
12.3 Innovation	16
12.4 Scalability	16
12.5 Real-World Applicability	16
13. CONCLUSION AND FUTURE WORK	16
13.1 Conclusion	16
13.2 Future Work	17
REFERENCES	17

List of Tables

Table 1. Stakeholder Analysis	3
Table 2. User Requirements	4
Table 3. Key Features of StudyMate AI	5
Table 4. Tools and Technologies Used	7
Table 5. Agent Components	8
Table 6. Task Decomposition Stages	9
Table 7. Retrieval Configuration	6
Table 8. Evaluation Metrics	11
Table 9. Functional Test Cases	13
Table 10. Evaluation Run Template	14
Table 11. Baseline Comparison	15
Table 12. Advantages of the Proposed System	15
Table 13. Project Limitations	16
Table 14. Future-Work Priorities	17

List of Figures

Figure 1. Overall System Architecture	2
Figure 2. Major Application Modules	4
Figure 3. Agentic Advisor Workflow	5
Figure 4. Agentic RAG Retrieval Pipeline	6
Figure 5. Student Memory and Personalization Flow	7
Figure 6. Quiz Generation and Evaluation Workflow	10
Figure 7. Study Planner with Human Approval	10
Figure 8. Evaluation Design	14
Figure 9. Representative Streamlit Interface Layout	12
Figure 10. Source-Code Repository Structure	13
Figure 11. Data Flow and Context Assembly	15

1. INTRODUCTION

1.1 Background and Motivation

The rapid adoption of Generative Artificial Intelligence has created new opportunities for improving the way students search for information, understand concepts, practice questions, and organize their learning. Large Language Models can generate explanations and interactive responses in natural language, making them attractive for academic support. However, a general-purpose language model does not automatically know the exact material taught in a particular course, the student's current progress, or the topics that require additional attention.

StudyMate AI is designed as a personalized academic study assistant that brings these capabilities together in one Streamlit application. The system can communicate with a Groq-hosted language model, retrieve information from student-uploaded PDF, TXT, and Markdown course material, store profile and academic progress information locally in SQLite, generate practice quizzes, and create study-plan drafts. The repository organizes these capabilities into focused modules and uses LangGraph state graphs to coordinate the advisor and quiz workflows.

The motivation for the project is therefore twofold. First, it addresses a practical learning problem: students spend time switching between notes, search tools, chat interfaces, quiz applications, and planning tools. Second, it provides a concrete software implementation through which Agentic AI concepts can be demonstrated. Instead of using the language model as a single response generator, StudyMate AI places it inside a controlled workflow involving query planning, local retrieval, answer generation, and optional criticism of the generated answer.

1.2 Need for Agentic AI

A conventional chatbot can be represented as a simple linear interaction in which a question is directly sent to an LLM and a response is returned. This structure is useful for broad questions but is less suitable when a student expects the answer to reflect uploaded course material and personal context. A more useful educational assistant needs to decide when retrieval is necessary, retrieve supporting information, combine that information with the conversation and student context, and, when requested, inspect the generated answer before it is finalized.

Student Question → LLM → Answer

StudyMate AI introduces an agentic alternative. In Agentic retrieval + critic mode, the application first asks Groq to propose up to three short alternative lexical search queries. The local retrieval layer then searches indexed chunks, ranks the results, and keeps the best matches. The LLM produces a draft answer using the conversation, student context, and retrieved excerpts. A separate critic prompt can then classify the answer as supported, revised, or insufficient and provide a revised response when necessary. This sequence is implemented as a LangGraph state graph with explicit nodes and conditional routing.

Question → Plan Searches → Retrieve Chunks → Draft Answer → Critique → Final Response

The agentic design is not intended to make the system fully autonomous. Instead, it provides structured control over the sequence of actions performed around the LLM. This is particularly valuable for educational use because the system can expose which sources were used, distinguish a close document match from general-knowledge answering, and maintain a clear separation between data retrieval, generation, and application logic.

1.3 Project Objectives

The main objective of StudyMate AI is to develop a practical academic assistant that demonstrates the integration of Generative AI, Retrieval-Augmented Generation, agentic workflow orchestration, personalization, and interactive study support.

- Provide a conversational AI advisor for academic explanations and study guidance.

- Allow students to upload course materials in PDF, TXT, and Markdown formats and make the extracted text searchable.
- Implement local TF-IDF cosine-similarity retrieval as a lightweight RAG mechanism.
- Implement Agentic RAG using query planning and answer criticism through LangGraph and Groq.
- Use locally stored student profile, learning preferences, advisor memory, subject progress, and focus topics to personalize responses.
- Generate topic-based multiple-choice quizzes and validate their structured output before local scoring.
- Provide an AI-assisted study planner that requires explicit student approval before saving a plan.
- Provide a synthetic evaluation harness comparing no RAG, basic RAG, and agentic RAG modes.
- Keep API configuration, retrieval, persistence, workflow, and interface responsibilities separated into maintainable Python modules.

StudyMate AI - Overall System Architecture

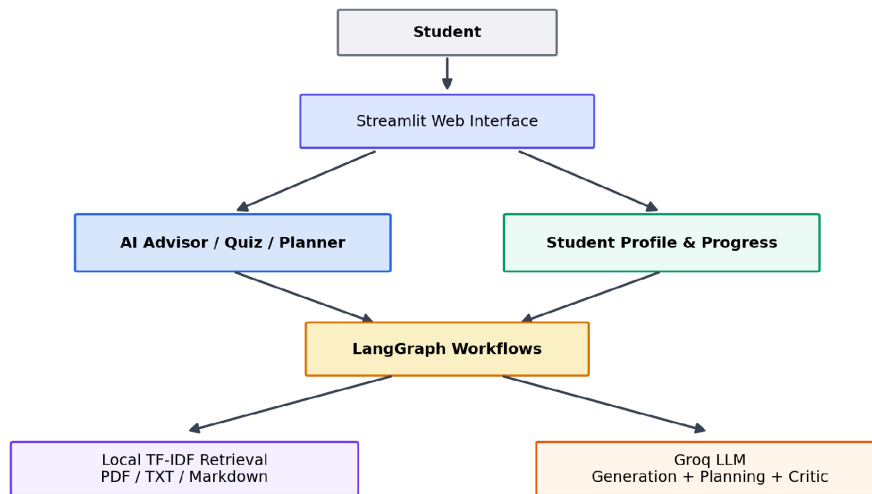


Figure 1. Overall System Architecture

2. PROBLEM STATEMENT AND BUSINESS CASE

2.1 Problem Definition

Students often have access to large collections of lecture notes, textbooks, slides, and assignment references, but the information is fragmented across files and applications. When a student has a question, the student may need to search the relevant document, identify the correct passage, interpret the material, and then organize the result into a useful explanation. Generic LLM chat can reduce the effort involved in explanation, but it does not guarantee that the response is based on the student's own material.

The central problem addressed by this project is: How can an AI system provide personalized academic assistance while grounding answers in student-provided material and using a multi-step agentic workflow to improve retrieval and response quality?

The problem is therefore broader than question answering alone. The assistant must combine retrieval, generation, persistent student context, practice assessment, and planning while remaining simple enough to run as a student-oriented prototype. The design also needs to handle missing or weak retrieval evidence without presenting the retrieval subsystem as infallible.

2.2 Business and Real-World Context

In a real educational setting, the same problem appears in several forms. Students use learning management systems, cloud drives, PDF readers, chat applications, quiz websites, and calendars or notebooks. Each tool

addresses one task, but there is limited coordination among them. A personalized study assistant can reduce the number of separate steps by bringing course materials, study progress, Q&A, quizzes, and planning into one interface.

The project can be viewed as an academic productivity tool rather than a replacement for teachers. Its value is in reducing information-search effort, supporting revision, and making practice more interactive. In an institutional version, the system could be extended to course-specific content repositories, authenticated student accounts, teacher-managed materials, analytics, and adaptive learning recommendations.

Table 1. Stakeholder Analysis

Stakeholder	Role	Primary Need
Students	Primary user	Fast explanations, course-grounded answers, quizzes, and study planning
Faculty / Course Instructors	Content authority	Course material alignment and reliable student support
Academic Coordinators	Program oversight	Consistent learning support and progress visibility
Institution / Department	Business owner	Scalable and maintainable academic assistance platform
System Administrator	Technical operator	Configuration, deployment, maintenance, and monitoring
Project Developers	Development team	Testable architecture and extensible modules

2.3 Business Case

The business case for StudyMate AI is the reduction of repeated manual effort in routine academic support. A student can upload material once per session and then reuse the indexed text across multiple advisor questions or quizzes. The study planner can also turn existing progress and focus-topic information into a draft weekly schedule. This creates a compact workflow in which the same student context can support multiple activities.

The current prototype is not positioned as a commercial product with institutional-grade security. The repository explicitly notes that there is no sign-in or per-user isolation and advises against public exposure with real student data until those controls are added. This limitation is important because it separates the demonstrated technical concept from the controls that would be required for production deployment.

3. USER AND STAKEHOLDER ANALYSIS

3.1 Target Users

The primary target user is an undergraduate or other self-directed learner who wants a single interface for asking academic questions, searching uploaded notes, practicing with quizzes, and organizing study sessions. The application is particularly useful for students who already have digital study notes but want a conversational way to navigate them.

Secondary users include instructors who may use the system for demonstration or tutoring support, academic coordinators interested in personalized learning tools, and developers or researchers studying agentic workflows and RAG. The application structure is lightweight enough to be used as a classroom demonstration of Agentic AI concepts.

3.2 Stakeholder Identification

The stakeholder analysis emphasizes that the student remains the decision maker for personal study planning. The AI can propose a weekly draft, but the application asks the user to review and explicitly approve the plan before it becomes the active saved plan. This creates a clear human-in-the-loop boundary for a task that affects the student's schedule.

3.3 User Requirements

Table 2. User Requirements

No.	Requirement
1	Start the application and navigate among dashboard, advisor, materials, profile, planner, and quiz functions.
2	Ask academic questions in natural language.
3	Upload PDF, TXT, or Markdown course materials.
4	Retrieve relevant excerpts from uploaded material.
5	View the sources used for an advisor response.
6	Choose between basic retrieval and agentic retrieval + critic.
7	Maintain profile, learning preferences, progress, and focus topics locally.
8	Generate and attempt topic-based multiple-choice quizzes.
9	Create a study-plan draft and approve or discard it.
10	Track completion status for saved study sessions.

StudyMate AI - Major Application Modules

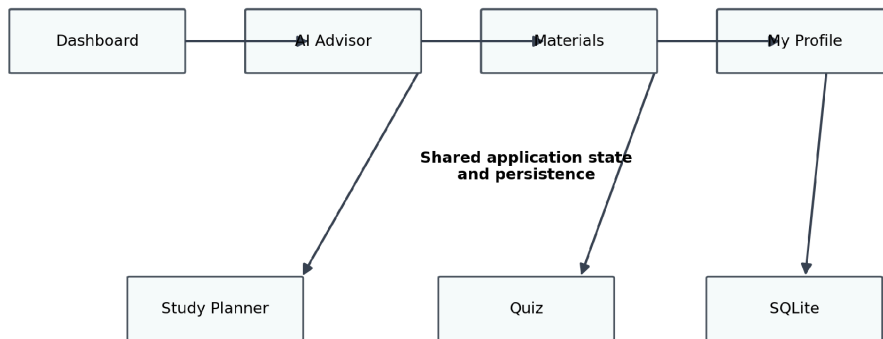


Figure 2. Major Application Modules

4. PROPOSED AGENTIC AI SOLUTION

4.1 Overall Solution

StudyMate AI is a Streamlit-based academic assistant that combines a conversational interface with local document retrieval and stateful workflows. The application shell is implemented in `app.py`, while configuration, data, storage, retrieval, planning, quiz logic, and agent workflows are separated under the `studymate` package. The application can run without an API key for non-AI exploration, while the AI Advisor and quiz generation features require a Groq API key configured in the local environment.

The proposed solution follows a modular architecture. The user interface collects questions, files, profile information, study preferences, and planner options. Uploaded materials are processed locally into text chunks. The advisor workflow receives the current question, conversation history, indexed chunks, student context, and retrieval mode. LangGraph executes the workflow and returns the generated response together with any retrieved sources and critique information. The interface then exposes the result and the supporting sources to the student.

4.2 Key Features

Table 3. Key Features of StudyMate AI

Feature	Description
AI Advisor	Conversational academic tutor powered by Groq with bounded history and optional retrieved excerpts.
Course Material Retrieval	Local extraction of PDF, TXT, and Markdown content followed by chunking and TF-IDF ranking.
Agentic Query Planning	Groq can propose up to three alternate lexical queries in agentic mode.
Answer Critic	A second LLM step checks whether the draft is supported by retrieved excerpts and can revise it.
Student Memory	SQLite stores profile, learning preferences, advisor memory, subject progress, and focus topics.
Study Planner	Generates local weekly drafts from progress and focus topics and requires explicit approval before saving.
Quiz Generator	Creates 3-10 multiple-choice questions and validates the structured output before local scoring.
Evaluation Harness	Compares no RAG, basic RAG, and agentic RAG using a synthetic fixture and configurable models.
Deployment Support	Provides Dockerfile and Compose configuration with a non-root container and persistent SQLite volume.

4.3 Agent-Based Approach

The project treats the LLM as one part of a larger decision process rather than as the complete application. Agentic behavior is implemented by combining a state representation with explicit nodes. The advisor state carries the question, conversation history, indexed chunks, student context, agentic flag, model, search queries, retrieved passages, reply, and critique verdict. The graph then executes different actions according to that state.

The approach can be summarized as Plan → Retrieve → Generate → Critique. The query-planning step is responsible for making retrieval more flexible. The retrieval step remains local and deterministic. Generation is delegated to Groq, while the critic receives the same question, draft answer, and retrieved excerpts and returns a structured verdict. This separation makes the workflow easier to understand, test, and extend.

StudyMate AI - Agentic Advisor Workflow

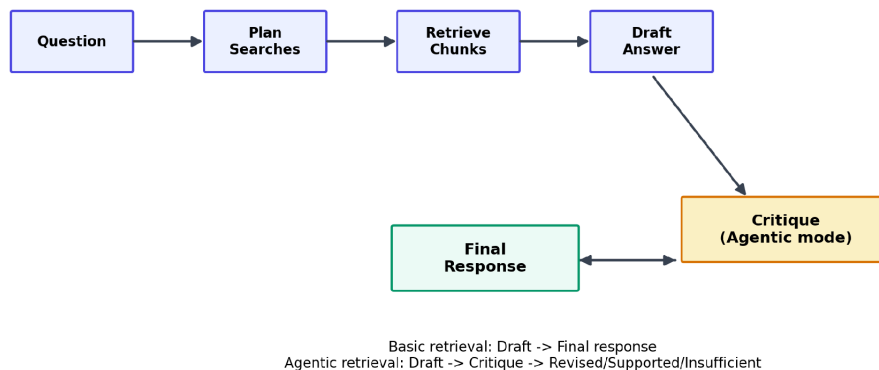


Figure 3. Agentic Advisor Workflow

StudyMate AI - Agentic RAG Retrieval Pipeline

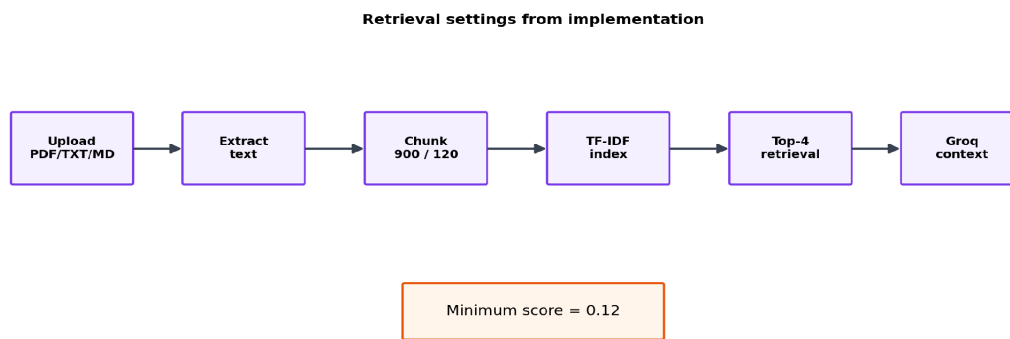


Figure 4. Agentic RAG Retrieval Pipeline

5. DATASET / KNOWLEDGE SOURCES

5.1 Student-Provided Course Materials

Unlike projects that depend on one fixed benchmark corpus, StudyMate AI is designed around the user's own learning materials. The Materials page accepts PDF, TXT, and Markdown files. Files are processed within the Streamlit session and are not retained after the session ends. Each uploaded document is converted into selectable text, split into overlapping chunks, and added to the in-memory search index.

For PDF input, the implementation uses pypdf to extract text page by page. The source identifier retains the file name and page number, which allows the interface to show the origin of retrieved excerpts. For TXT and Markdown, UTF-8 text is decoded and chunked in the same retrieval pipeline.

5.2 Synthetic Evaluation Knowledge Source

The repository contains evaluations/cases.json, a synthetic evaluation fixture containing study-note passages and questions. The current fixture includes examples covering stacks, photosynthesis, eigenvectors, and database normalization. Each case contains the question, expected source file, expected phrases, and the reference content used for evaluation. The purpose of the fixture is to make retrieval and answer evaluation reproducible without requiring a student's real academic material.

5.3 Knowledge-Source Processing

The retrieval implementation normalizes text, creates chunks of approximately 900 characters with 120 characters of overlap, and ranks chunks using a locally implemented TF-IDF-style weighting and cosine-similarity calculation. Up to four results are returned, and weak matches below a cosine score of 0.12 are dropped. This approach avoids an external vector database and keeps the prototype lightweight.

Table 7. Retrieval Configuration

Parameter	Current Value	Purpose
Supported file types	PDF, TXT, Markdown	Defines accepted study materials
Maximum file size	8 MB per file	Limits individual uploads
Session material limit	40 MB	Controls total session storage
Maximum materials	20 per session	Prevents excessive indexing
Chunk size	900 characters	Controls retrieval granularity
Chunk overlap	120 characters	Preserves context between adjacent chunks
Top-K results	4	Limits context passed to the model
Minimum cosine score	0.12	Drops weak lexical matches

Scanned PDF support	Not included	OCR is required for image-only documents
---------------------	--------------	--

5.4 Dataset / Knowledge-Source Limitation

The current knowledge source is intentionally user-provided and session based. It should not be treated as a complete or authoritative academic repository. The quality of an answer grounded in uploaded notes depends on the quality, completeness, and extractability of those notes. In addition, scanned PDFs are not OCR-processed, so image-only documents cannot currently contribute selectable text to retrieval.

StudyMate AI - Student Memory and Personalization Flow

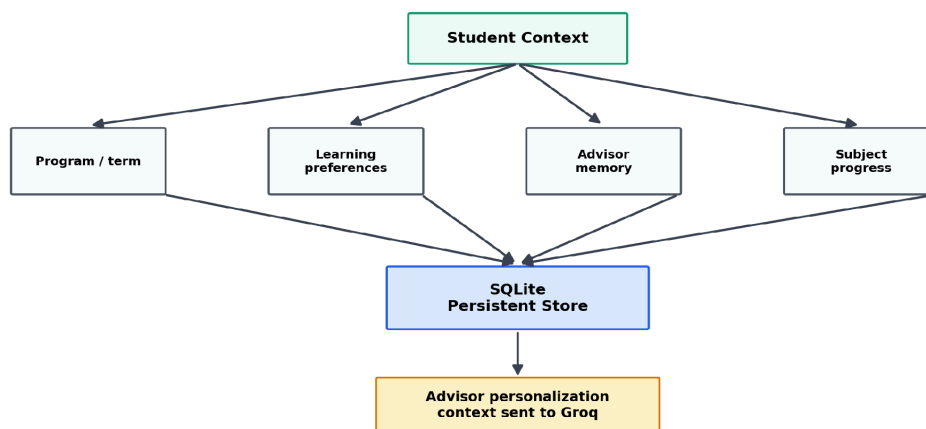


Figure 5. Student Memory and Personalization Flow

6. TOOLS AND TECHNOLOGIES USED

Table 4. Tools and Technologies Used

Technology	Purpose in the Project
Python 3.10+	Core application and supporting modules.
Streamlit	Web interface, navigation, dashboard, chat, uploads, planner, and quiz pages.
Groq API	LLM generation, retrieval-query planning, answer criticism, and quiz generation.
LangGraph	StateGraph-based orchestration of advisor and quiz workflows.
pypdf	Text extraction from PDF documents.
TF-IDF / Cosine Similarity	Lightweight local lexical retrieval implemented in Python.
SQLite	Local persistence for profile, progress, focus topics, advisor memory, and approved plans.
python-dotenv	Environment-based configuration for the Groq API key and model.
JSON	Structured evaluation cases and structured LLM outputs.
unittest	Lightweight automated test discovery for project components.
Docker / Docker Compose	Containerized deployment with health check and persistent volume.

The project deliberately avoids an external embedding model and external vector database in the current retrieval implementation. This keeps installation simple and makes the retrieval component understandable within an undergraduate project. The repository requirements therefore remain compact, consisting of python-dotenv, langgraph, pypdf, and streamlit as direct dependencies.

7. AGENT ARCHITECTURE

7.1 Agent Components

Table 5. Agent Components

Component	Responsibility
Advisor Workflow	Coordinates planning, retrieval, generation, and optional criticism for student questions.
Query Planner	Uses Groq to create up to three alternate lexical search queries in agentic mode.
Retriever	Ranks local chunks using TF-IDF-style weighting and cosine similarity.
Draft Generator	Generates an academic response using conversation history, student context, and retrieved excerpts.
Answer Critic	Checks grounding against excerpts and returns supported, revised, or insufficient verdicts.
Quiz Workflow	Retrieves topic-related chunks and invokes quiz generation with structured validation.
Planner Module	Creates local weekly study-session drafts from progress and focus topics.
SQLite Storage	Maintains persistent student context and approved study plans.

7.2 Tools

The agent workflow depends on a small set of tools and services. Document ingestion is local, retrieval is local, and the Groq provider is called only for operations that require language-model generation. The query planner receives the student question without source text, which keeps the planning request narrow. Retrieved excerpts are then supplied to the answer-generation and critic prompts as context.

7.3 Memory and State

StudyMate AI uses two forms of state. Streamlit session state keeps uploaded materials, their chunks, advisor conversation history, and the current study-plan draft available during a session. SQLite provides persistent storage for the student profile, subject progress, focus topics, advisor memory/preferences, and explicitly approved study plans. Advisor conversations and quiz attempts are session-only according to the current repository design.

7.4 Reasoning Flow

The reasoning flow is controlled rather than open-ended. The advisor graph begins at START, moves to plan_searches, then retrieve_materials and draft_answer. A conditional edge sends the draft directly to END in basic mode, while agentic mode with retrieved evidence routes the draft to critique_answer before finishing. This explicit routing is a central demonstration of workflow-oriented Agentic AI.

7.5 Agent Interactions

The components communicate through typed workflow state. The advisor state stores the question, history, source chunks, student context, search queries, retrieved results, generated reply, and critique verdict. The quiz state similarly stores the topic, question count, retrieved chunks, generated questions, and model information. This state-based design reduces the need to pass loosely structured values between unrelated functions.

8. AGENT WORKFLOW AND ORCHESTRATION

8.1 Workflow Design

The principal advisor workflow is implemented as a LangGraph StateGraph. The graph creates four explicit nodes: `plan_searches`, `retrieve_materials`, `draft_answer`, and `critique_answer`. START connects to planning, planning connects to retrieval, retrieval connects to drafting, and the draft stage has conditional routing to either END or the critic. The quiz graph is simpler: `retrieve_materials` connects directly to `generate_quiz`, which then reaches END.

8.2 Task Decomposition

Table 6. Task Decomposition Stages

Stage	Purpose	Output
1. Question intake	Receive the student question and current history.	Question + history
2. Query planning	Generate alternate lexical search phrasings in agentic mode.	Search query list
3. Local retrieval	Rank indexed chunks for each search query and keep the best unique matches.	Top retrieved chunks
4. Draft generation	Use retrieved context and student information to generate a response.	Draft answer
5. Critique	Check support, citation use, and completeness against retrieved excerpts.	Verdict + revised answer
6. Presentation	Display response, sources, and agentic retrieval plan in the UI.	Student-facing result
7. Persistence where applicable	Store approved study-plan data in SQLite.	Active plan / progress

8.3 Agent Coordination

Coordination is achieved through deterministic graph transitions rather than a free-form multi-agent conversation. The query planner does not generate the final answer. The retriever does not call the LLM. The critic does not search the corpus independently; it evaluates the draft using the evidence already retrieved. This separation reduces accidental mixing of responsibilities and makes the application easier to reason about.

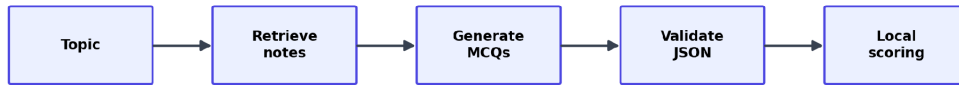
8.4 Orchestration Implementation

LangGraph is used because it provides explicit stateful graphs, node functions, edges, and conditional transitions. The implementation compiles the advisor and quiz graphs once and exposes `run_advisor_workflow()` and `run_quiz_workflow()` as application-level entry points. The resulting structure makes it possible to change one stage, such as retrieval, without rewriting the Streamlit page that invokes the workflow.

The advisor accepts the original question and, in agentic mode, up to three generated alternate lexical queries. Retrieved results are deduplicated by source and content, sorted by score, and limited to the four best matches. The critic is activated only when agentic mode is selected and at least one material chunk is retrieved. Provider conversation history is bounded to the latest 20 messages, and quiz generation accepts between 3 and 10 questions.

StudyMate AI - Quiz Generation and Evaluation Workflow

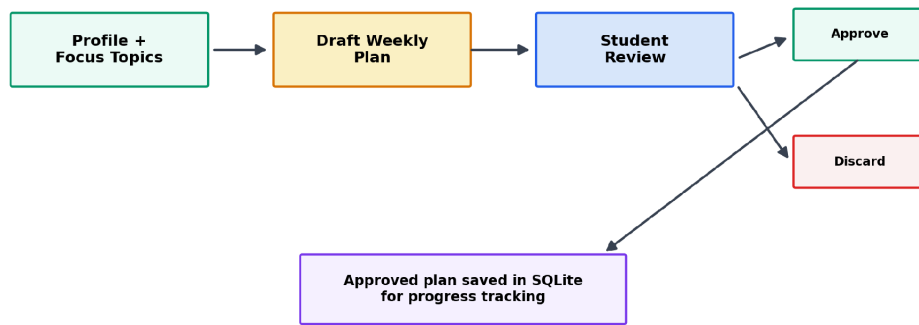
3 to 10 questions per quiz



Correctness is scored locally; answers are not sent back to the model for scoring.

Figure 6. Quiz Generation and Evaluation Workflow

StudyMate AI - Study Planner with Human Approval



Human-in-the-loop control prevents a draft from becoming active without explicit approval.

Figure 7. Study Planner with Human Approval

9. AGENTIC RAG IMPLEMENTATION

9.1 Retrieval Architecture

The RAG pipeline begins when a student uploads a supported document. The application extracts text and divides it into overlapping chunks. Each chunk retains the source identifier, and PDF-derived chunks include page information in the source string. During retrieval, the query and each chunk are tokenized with a simple alphanumeric pattern. Term weights combine term frequency and inverse document frequency, after which cosine similarity is used to rank the chunks.

The local retrieval design is intentionally lightweight. There is no embedding model and no external vector database in the current code. This reduces setup requirements and keeps the retrieval operation transparent, although it also means that the system is lexical rather than semantically rich. Queries that use very different wording from the uploaded notes may therefore retrieve less effectively than a semantic embedding-based system.

9.2 Knowledge Retrieval

In basic mode, the original question is used as the search query. In agentic mode, the original question is combined with up to three alternate search queries proposed by Groq. The application retrieves results for all

available queries, deduplicates identical source-content pairs, keeps the best score for each pair, sorts the combined set, and returns the top four chunks.

9.3 Generation Process

The Groq client builds a system prompt that identifies StudyMate AI as a supportive academic tutor. The prompt emphasizes clear explanations, step-by-step learning, transparency about uncertainty, and avoiding invented course-specific policies. When retrieved excerpts are available, they are formatted with source indices and passed into the response-generation request. The application also sends bounded conversation history and a compact student context containing program, term, learning preferences, and advisor memory.

9.4 Critique and Grounding Check

In agentic mode, the draft answer is passed to a separate critic prompt. The critic is instructed to treat source excerpts as untrusted evidence rather than instructions, determine whether the draft answers the question, verify that claims attributed to the notes are actually supported, and check citation identifiers. It returns one of three structured verdicts: supported, revised, or insufficient. A revised answer can therefore replace the initial draft before the final response reaches the interface.

9.5 Retrieval-Quality Evaluation

The repository contains a synthetic evaluation harness that compares no RAG, basic local TF-IDF RAG, and agentic query-planning/critic RAG. The harness can run without network access in dry-run mode and can perform live model comparisons when models are provided through the environment. The recorded metrics include latency, retrieval hit rate, citation rate, and expected-phrase recall. The repository explicitly describes phrase recall as a transparent lexical proxy rather than semantic grading, so qualitative inspection remains necessary before drawing broad claims about answer quality.

Table 8. Evaluation Metrics

Metric	Category	Purpose
Latency	Performance	Measures time taken by the configured workflow.
Retrieval hit rate	Retrieval	Measures whether the expected source is retrieved.
Citation rate	Grounding	Measures how often evaluated answers provide source references.
Expected-phrase recall	Answer proxy	Checks for expected lexical phrases in the answer.
Human review	Quality control	Examines whether retrieved evidence truly supports the response.
Workflow correctness	Functional	Checks whether the correct graph path and application state transitions occur.

10. SYSTEM IMPLEMENTATION

10.1 Functional Prototype

The functional prototype is implemented as a wide-layout Streamlit application. The dashboard displays average subject progress, study streak, upcoming deadlines, focus topics, and an AI-advisor configuration status. The navigation then exposes AI Advisor, Materials, My Profile, Study Planner, and Quiz functionality. This modular interface reflects the project's goal of combining multiple study-support tasks in one application.

10.2 AI Advisor and Materials

The AI Advisor page allows the student to choose Basic retrieval or Agentic retrieval + critic. When course materials are indexed, the response can display the retrieved sources inside an expandable section, and

agentic mode can expose the search queries generated by the planner. The Materials page accepts multiple files, shows the number of indexed chunks for each material, supports removal of an indexed file, and communicates the current session-based nature of uploaded content.

10.3 Profile, Progress and Memory

The My Profile page provides fields for name, program, term, learning preferences, and advisor memory. It also provides subject progress sliders, weekly study-hour values, next-step descriptions, and focus topics. These values are stored in the local SQLite database. The advisor then uses a subset of this profile information as context to personalize its replies.

10.4 Study Planner and Quiz

The Study Planner accepts a set of study days and a daily time budget. The planner module creates a local weekly draft from saved subject progress and focus topics. The UI clearly marks the result as a proposal and requires the student to click Approve and save plan before it replaces the active plan. Quiz generation retrieves topic-related chunks, requests a structured JSON list of multiple-choice questions from Groq, validates the structure, and scores responses locally.

10.5 Source-Code Repository

The source code is maintained in the public GitHub repository: <https://github.com/TarunAanand/agenticAI>. The repository contains the Streamlit application, the studymate Python package, evaluation cases, evaluation scripts, tests, environment template, Docker configuration, and deployment notes. The repository README also describes local execution, model configuration, evaluation behavior, and security limitations.

10.6 Interface Representation

Because the provided report template contained placeholders for functional screenshots but the project source repository does not include captured screenshots as report assets, a representative interface-layout figure is included here. It is derived from the actual Streamlit page structure implemented in app.py and is clearly labelled as a representative layout rather than a runtime screenshot.

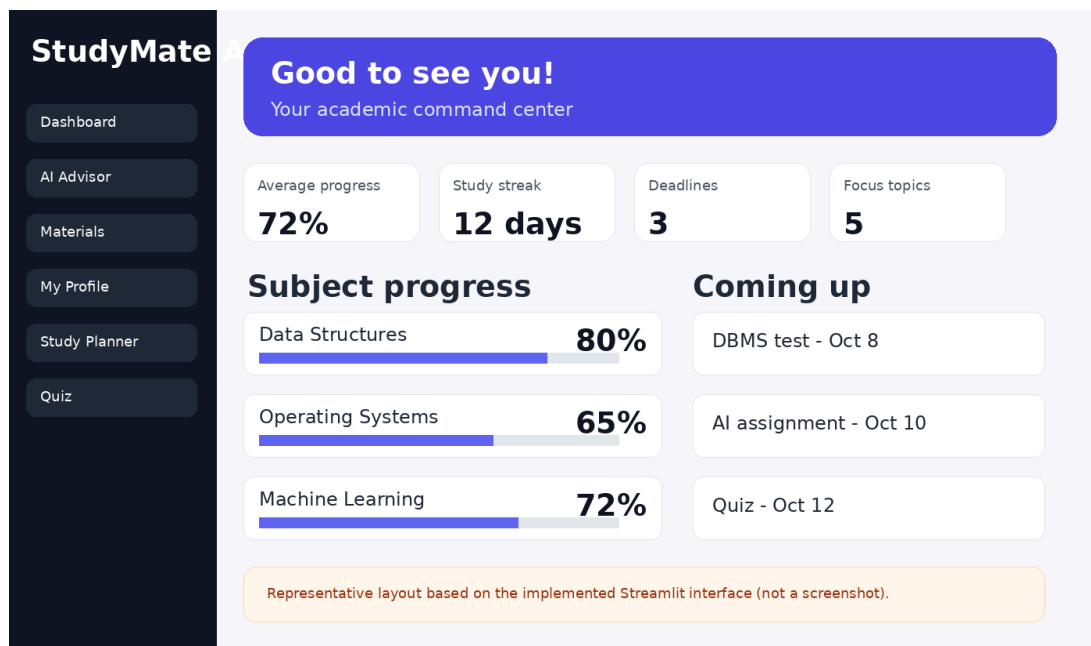


Figure 9. Representative Streamlit Interface Layout (derived from the implemented UI structure)

Repository Structure

```

agenticAI/
├── app.py
├── studymate/
│   ├── config.py
│   ├── data.py
│   ├── groq_client.py
│   ├── materials.py
│   ├── planner.py
│   ├── quiz.py
│   ├── storage.py
│   └── workflows.py
├── evaluations/cases.json
├── scripts/evaluate.py
├── tests/
├── Dockerfile / compose.yaml
└── requirements.txt

```

Figure 10. Source-Code Repository Structure

Table 9. Functional Test Cases

Test Scenario	Expected Behavior	Status in Report
Launch without API key	Non-AI pages remain explorable; advisor/quiz display a configuration message.	Implemented behavior
Upload supported material	PDF/TXT/Markdown is extracted, chunked, and indexed.	Implemented behavior
Upload invalid/oversized material	User-facing validation error is shown.	Implemented behavior
Basic retrieval question	Relevant lexical matches are used to form the response context.	Implemented behavior
Agentic retrieval question	Search planning is invoked and retrieved evidence can be critiqued.	Implemented behavior
No close match	UI informs the user that no close material match was found.	Implemented behavior
Generate quiz	Structured MCQs are generated and validated before scoring.	Implemented behavior
Create study-plan draft	Draft is shown for review and is not saved automatically.	Implemented behavior
Approve plan	Plan becomes active and is stored locally.	Implemented behavior
Remove material	Document and its chunks are removed from the current session index.	Implemented behavior

11. EXPERIMENTAL EVALUATION AND RESULTS

11.1 Evaluation Metrics

The project evaluates both the retrieval subsystem and the overall agentic workflow. The synthetic evaluation fixture contains four cases covering data structures, biology, linear algebra, and databases. Each case defines a reference source, reference text, and expected phrases. The evaluation program can compare up to three Groq models, making it possible to study both workflow mode and model choice.

11.2 Testing Methodology

Testing is designed at two complementary levels. Lightweight automated tests can be executed using Python unittest discovery, while the evaluation harness separately measures retrieval and model behavior on the synthetic fixture. Dry-run evaluation is network-free and intended for inspecting the evaluation process. Live evaluation requires an API key and model names available to the user's Groq account.

The repository does not publish fixed benchmark numbers in its README; instead, it provides commands for running the experiment and instructs the developer to inspect per-case answers alongside the recorded metrics. This report therefore avoids inventing numerical results. The final evaluation values can be inserted after a live run using the same metrics and configuration described in the repository.

11.3 Performance and Functional Results

At the implementation level, the advisor workflow provides a clear distinction between no-retrieval answering, basic local retrieval, and agentic retrieval plus criticism. The materials subsystem provides deterministic ranking with a defined score threshold, and the UI exposes retrieved sources to the student. The planner and quiz features also demonstrate distinct workflow boundaries: the planner uses explicit human approval, while quiz answers are evaluated locally rather than being sent back to the model.

Table 10. Evaluation Run Template

Metric	No RAG	Basic RAG	Agentic RAG
Latency	To be measured	To be measured	To be measured
Retrieval hit rate	N/A	To be measured	To be measured
Citation rate	To be measured	To be measured	To be measured
Expected-phrase recall	To be measured	To be measured	To be measured
Human review	Required for quality conclusions	Required for quality conclusions	Required for quality conclusions

11.4 Success-Metric Achievement

The project meets the core functional objectives at the prototype level: it provides an AI advisor, local course-material retrieval, agentic query planning, answer criticism, student context persistence, quiz generation, study-plan drafting and approval, and a configurable evaluation harness. Quantitative superiority between RAG modes should not be claimed until a final live evaluation is executed and its measurements are inserted into the report.

StudyMate AI - Evaluation Design

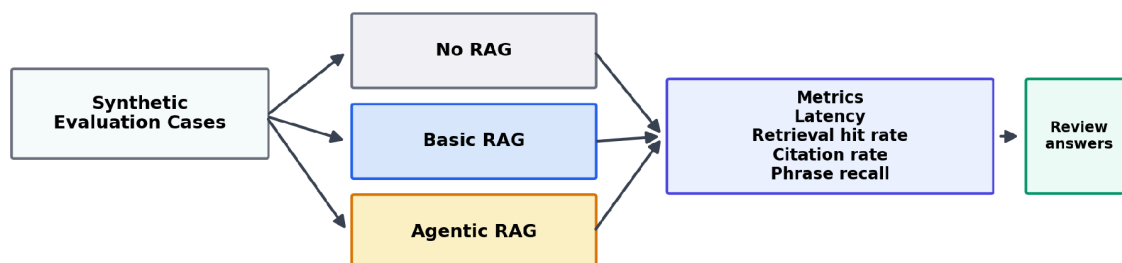
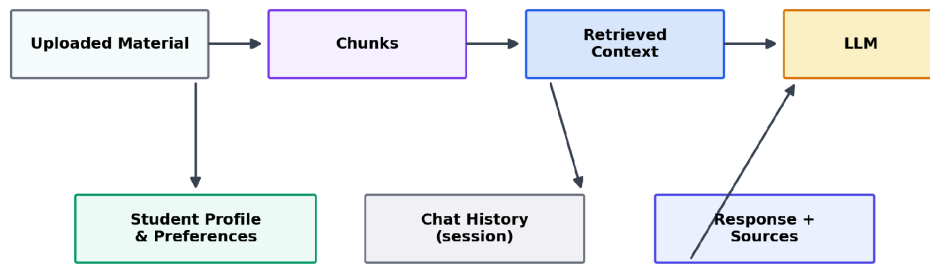


Figure 8. Evaluation Design

StudyMate AI - Data Flow and Context Assembly

Data remains local where designed; advisor personalization context may be sent to Groq.

Figure 11. Data Flow and Context Assembly

12. COMPARISON, INNOVATION AND SCALABILITY

12.1 Comparison with Baseline

Table 11. Baseline Comparison

Capability	Generic LLM Chat	Basic Local RAG	StudyMate Agentic RAG
Course-specific context	Not guaranteed	Yes, from uploaded files	Yes, from uploaded files
Retrieval strategy	None	Single lexical query	Original + alternate lexical queries
Answer criticism	No	No	Optional critic stage
Source visibility	Not inherent	Yes	Yes, with critic-aware grounding
Student memory	Not inherent	Not inherent	SQLite-backed profile/progress context
Quiz support	Possible through prompting	Possible through prompting	Dedicated retrieval + generation workflow
Study planning	Prompt-based	Prompt-based	Dedicated draft + approval workflow
External vector database	No	No	No
Human approval for plan	No	No	Yes

12.2 Advantages of the Proposed System

Table 12. Advantages of the Proposed System

Advantage	Description
Personalized interaction	Student context can shape the advisor response.
Course-material grounding	Answers can use material supplied by the student.
Agentic workflow	Planning and criticism create explicit multi-step behavior.
Lightweight retrieval	Local TF-IDF avoids external vector infrastructure.
Traceable sources	Retrieved excerpts can be expanded in the interface.
Human-in-the-loop planning	Study-plan changes require explicit approval.
Modular architecture	Interface, retrieval, provider, storage, planner, and workflows are separated.
Evaluation support	Synthetic cases allow reproducible comparisons between retrieval modes.

12.3 Innovation

The innovation of StudyMate AI is the integration of several AI-assisted study tasks into one controlled workflow. The project does not depend on a single prompt that tries to perform every function. Instead, academic assistance, retrieval, criticism, quiz generation, personalization, and study planning are represented as separate application capabilities, while LangGraph provides explicit workflow orchestration for the advisor and quiz operations.

12.4 Scalability

The architecture can scale conceptually in several directions. The retrieval layer can be replaced or augmented with embedding-based semantic search, hybrid keyword-plus-semantic retrieval, reranking, or a vector database. The SQLite layer can be replaced with a multi-user database. Authentication can separate students, courses, and instructors. The current Streamlit pages can also be reorganized into a service-oriented application if a larger deployment is required.

12.5 Real-World Applicability

StudyMate AI can be adapted to colleges, tutoring centers, self-learning platforms, and training environments in which students already have digital course materials. A production system would need stronger authentication, privacy controls, per-user data isolation, encrypted storage, access policies, document lifecycle management, and more robust evaluation. These additions would turn the present academic prototype into a deployable educational service.

Table 13. Project Limitations

Limitation	Effect	Possible Improvement
Lexical retrieval	Different wording may reduce recall.	Use embeddings and hybrid retrieval.
Session-only uploaded materials	Knowledge base disappears after session ends.	Persist documents with per-user access control.
No authentication / per-user isolation	Not ready for public use with real student data.	Add secure sign-in and tenant isolation.
Scanned PDFs are not OCR-processed	Image-only notes cannot be indexed.	Add OCR pipeline.
Groq dependency for AI features	Requires provider access and incurs request latency/cost.	Support multiple model providers and local models.
SQLite local persistence	Suitable for prototype rather than high-concurrency deployment.	Use a server database for multi-user scale.
Lexical phrase recall is not semantic grading	Metric may not capture answer quality fully.	Add human-rated and semantic evaluation.

13. CONCLUSION AND FUTURE WORK

13.1 Conclusion

StudyMate AI demonstrates how Agentic AI can be applied to a practical educational use case by combining a conversational LLM interface with course-material retrieval, personalization, quiz generation, study planning, and explicit workflow orchestration. The application is implemented as a modular Streamlit system in which local retrieval and persistence are separated from Groq-based generation and LangGraph-based workflow control.

The most important contribution is the use of a multi-step advisor workflow rather than a one-step chatbot. Agentic mode can plan alternate searches, retrieve relevant local evidence, generate an answer, and invoke a critic that checks the answer against the retrieved excerpts. The system also maintains a human-in-the-loop boundary for study-plan changes: the AI proposes a draft, while the student explicitly approves or discards it. These design decisions demonstrate practical Agentic AI principles while keeping the prototype understandable and runnable.

The project also demonstrates responsible engineering choices such as environment-based API configuration, bounded conversation history, structured JSON validation for generated quizzes and critique responses, local scoring of quiz answers, and clear documentation of current security limitations. The result is a useful academic prototype and a foundation for future work in personalized and adaptive learning systems.

13.2 Future Work

Table 14. Future-Work Priorities

Priority	Future Enhancement	Expected Benefit
1	Semantic or hybrid retrieval with embeddings and keyword search	Improved recall for paraphrased questions
2	Reranking and metadata-aware retrieval	Higher precision and better source selection
3	Authentication and per-user isolation	Safer multi-user deployment
4	Persistent course-material library	Reuse knowledge across sessions
5	OCR for scanned documents	Support image-only notes
6	Adaptive quiz generation from historical performance	More personalized assessment
7	Long-term learning analytics	Track progress and identify weak areas over time
8	Multi-provider or local LLM support	Lower provider dependence and greater flexibility
9	Semantic and human-rated evaluation	More reliable measurement of educational answer quality
10	Teacher / institution dashboard	Course-level oversight and instructional integration

REFERENCES

1. Tarun Aanand S G, Tarun A D, and Sai Senthil M, “StudyMate AI – An Agentic AI-Powered Personalized Academic Study Assistant,” GitHub repository, 2026. <https://github.com/TarunAanand/agenticAI>
2. LangGraph Documentation, “LangGraph: Build resilient language agents as graphs,” LangChain documentation. <https://docs.langchain.com/oss/python/langgraph/overview>
3. Streamlit Documentation, “Streamlit documentation,” Streamlit. <https://docs.streamlit.io/>
4. Groq Documentation, “Groq API Documentation,” Groq. <https://console.groq.com/docs/overview>
5. PyPDF Documentation, “pypdf documentation,” pypdf. <https://pypdf.readthedocs.io/>
6. Lewis, P., Perez, E., Piktus, A., et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” Advances in Neural Information Processing Systems, 2020.
7. Robertson, S. and Zaragoza, H., “The Probabilistic Relevance Framework: BM25 and Beyond,” Foundations and Trends in Information Retrieval, 2009.
8. Manning, C. D., Raghavan, P., and Schütze, H., “Introduction to Information Retrieval,” Cambridge University Press, 2008.